

プログラミングI 演習

Computer Programming I: Exercises

第11回 ファイル入出力

担当

A-P1 辻 愛里

A-P2 北 直樹

A-P3 清水 昭伸

本日の講義

本日の目標

- ファイルから値を入力できるようになろう
- プログラムの実行結果をファイルに出力できるようになろう

本日の資料

1. 前回のおさらい
2. ファイル入出力
3. 演習課題
4. 第2回レポート課題について

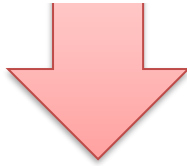
1. 前回のおさらい

計算機の中の文字

計算機の中には数値しか存在しない



人間にとっては、文字も扱いたい



文字(画像データ)に番号(数値)を割り振って管理する仕組みを作ればいい

文字に割り振られた番号を「文字コード」と呼ぶ

ここでは、基本的な文字コードの集合であるJIS X 0201コードを扱う

JIS X 0201 文字コード表

文字	数値	文字	数値
+	43	W	87
,	44	X	88
-	45	Y	89
.	46	Z	90
/	47	[	91
0	48	¥	92
1	49	]	93

キーボードに書かれているような、
いわゆる半角文字

1. 前回のおさらい

文字列

数列が「int型の配列」で表せるのだから……

文字列は「char型の配列」として実現できる

char型の配列を使ったプログラム

```
#include <stdio.h>
int main(void) {
    char String[20]={'H','e','l','l','o','W','o','r','l','d','\0'};
    int i;
    for(i=0;i<20;i++)
        printf("%c",String[i]);
    printf("\n");
    return 0;
}
```



毎回文字の長さに応じた処理を作っているのは扱いにくい上、このコードでは文字列の終端のヌル文字（'\0'）以降も強制的に出力してしまう。初期化されていない配列の場合には入っている値が不明で、それまで表示される点で良くない。

実行結果

HelloWorld

1. 前回のおさらい

文字列の入出力 (第9回のスライドより)

```
int main(void)
{
    char String[20];
    scanf("%19s",String);
    printf("%s\n",String);
    return 0;
}
```

配列名が先頭のアドレスを記録したポインタを表すので'&'をつけなくて良い(第7と8回で学習済み)

関数「scanf」も「%s」を用いて文字列を入力することができる。

(注: 配列の要素数より一つ少ない数を用いて%19sとすることでユーザの入力文字数が宣言した配列の要素数以上になった場合のバッファオーバーランを防ぐことができる)

printfも%sなら配列名を引数に書く
(第10回で説明)

→%sの位置にStringの文字列を表示

1. 前回のおさらい

演習10-3 文字列の入力

```
#include<stdio.h>
int main(void){
    char String[20];
    int i=0;

    scanf("%19s", String);

    while(String[i] != '¥0'){
        if(String[i] >= 'a' && String[i] <= 'z')
            String[i] = String[i] + ('A' - 'a');
        i++;
    }

    printf("%s¥n", String);
    return 0;
}
```

このとおりプログラムし実行し、小文字が大文字に変わることを確認しよう。

また、大文字をすべて小文字にするプログラムに書き換えてみよう

この部分の解答例は次のスライド

JIS X 0201コードを確認して、この操作の意味を確認してみてください。文字を数値に変換して考えてみよう

1. 前回のおさらい

演習10-3 解答例

```
#include<stdio.h>
int main(void){
    char String[20];
    int i=0;

    scanf("%19s", String);

    while(String[i] != '\0'){
        if(String[i] >= 'A' && String[i] <= 'Z')
            String[i] = String[i] - ('A' - 'a');
        i++;
    }

    printf("%s\n", String);
    return 0;
}
```

'A'-'a'は-32です。
入力されたA~Zの32個後がa~zです。
この数は、コンピュータ内部での文字表現に依存します。

1. 前回のおさらい

演習10-5 パスワードチェック機能の作成

1. 入力されたパスワードが「良いパスワード」か否かを判定する
2. 「良いパスワード」と判定された場合、(確認のため)再入力を求める

良いパスワードの条件(調べればもっと良い条件がありますが、この演習では下記とします)

1. 半角で8文字以上
2. 数字(0~9), 小文字(a~z), 大文字(A~Z)のすべての種類が使われている. (例: abcdef → ×, abcdefGH → ×, abCDef43 → ○)

ヒント

- 入力文字数チェックは, strlen関数を使うと良い.
- 数字, 小文字, 大文字のチェックは演習10-3等を参考にすると良い.
- 再入力したものが最初と同じかどうかは, strcmp関数を使ってチェックすると良い.

1. 前回のおさらい

演習10-5 解答例前半(チェック関数①)

```
#include<stdio.h>
#include<string.h>
```

strlen関数やstrcmp関数を使うために必要

```
int NumCheck(char String[]){
```

文字列中に数字があるか否かのチェック

```
    int i;
    for(i=0; i<(int)strlen(String); i++)
        if(String[i] >= '0' && String[i] <= '9') return 1;
    printf(" ※数字がありません\n");
    return 0;
```

```
}
```

```
int UpperCheck(char String[]){
```

文字列中に大文字があるか否かのチェック

```
    int i;
    for(i=0; i<(int)strlen(String); i++)
        if(String[i] >= 'A' && String[i] <= 'Z') return 1;
    printf(" ※大文字がありません\n");
    return 0;
```

```
}
```

1. 前回のおさらい

演習10-5 解答例前半(チェック関数②)

```
int LowerCheck(char String[]){  
    int i;  
    for(i=0; i<(int) strlen(String); i++)  
        if(String[i] >= 'a' && String[i] <= 'z') return 1;  
    printf(" ※小文字がありません¥n");  
    return 0;  
}
```

文字列中に小文字があるか否かのチェック

```
int LengthCheck(char String[]){  
    if(strlen(String) >= 8) return 1;  
    else{  
        printf(" ※8文字以上ありません¥n");  
        return 0;  
    }  
}
```

文字列が8文字以上か否かのチェック

1. 前回のおさらい

演習10-5 解答例後半(main関数)

```
int main(void)
{
    char Password1[20], Password2[20];

    printf("登録するパスワードを入力:");
    while(1) {
        scanf("%19s", Password1);
        if (NumCheck(Password1) && UpperCheck(Password1)
            && LowerCheck(Password1) && LengthCheck(Password1))
            break;
        printf("もう一度, 登録するパスワードを入力してください:");
    }

    printf("確認のため, 今入力したパスワードをもう一度入力してください:");
    while(1) {
        scanf("%19s", Password2);
        if (strcmp(Password1, Password2) == 0) break;
        printf("今入力したものと違います. もう一度入力してください:");
    }
    printf("パスワードは正常に登録されました\n");
    return 0;
}
```

このif文でパスワードチェック
(数字, 大文字, 小文字, 長さ)

チェックを通過したら再確認

1. 前回のおさらい

発展10-1 名簿の整理プログラム

要求される機能

1. 5人分の姓・名を入力する（姓→名の順で入力するようにし、姓名間に1つ半角スペースをいれる。5人の姓名はプログラム中で設定しても良い）
2. 出力用に書式を揃えて表示する
→ 書式: 名（先頭は大文字, 他は小文字）（半角スペース）姓（全て大文字）

例: 入力=kOnDo saToshi → 出力= Satoshi KONDO

必要になりそうな機能

1. 「姓」を文字列をすべて大文字にする
2. 「名」文字列をすべて小文字にする。その後、先頭だけ大文字にする
3. 「名」から出力する

1. 前回のおさらい

発展10-1 解答例

```
#include <stdio.h>
int main(void)
{
    char NameList[5][2][20];
    int i,j,CharIndex;
    for(i=0;i<5;i++){
        printf("名前の入力:");
        scanf("%19s %19s", NameList[i][0],NameList[i][1]);
    }
    for(i=0;i<5;i++){
        CharIndex = 0;
        while(NameList[i][0][CharIndex] != '¥0'){
            if(NameList[i][0][CharIndex] >= 'a' && NameList[i][0][CharIndex] <= 'z')
                NameList[i][0][CharIndex] += ('A' - 'a');
            CharIndex++;
        }
        CharIndex = 0;
        while(NameList[i][1][CharIndex] != '¥0'){
            if(NameList[i][1][CharIndex]>='A' && NameList[i][1][CharIndex]<='Z')
                NameList[i][1][CharIndex] += ('a' - 'A');
            CharIndex++;
        }
        NameList[i][1][0] += ('A' - 'a');
        printf("%s %s¥n", NameList[i][1],NameList[i][0]);
    }
    return 0;
}
```

NameList[i][0]に姓がNameList[i][1]に名が入る

姓をすべて大文字

名をすべて小文字

NameList[i][1][0]→名の先頭を大文字

名→姓の順に出力

1. 前回のおさらい

発展10-1 別解(前のスライドとの違いを考えてみよう)

```
#include <stdio.h>
int main(void)
{
    char NameList[5][2][20];
    int i,j,CharIndex;
    for(i=0;i<5;i++){
        printf("名前の入力:");
        scanf("%19s %19s", NameList[i][0],NameList[i][1]);
    }
    for(i=0;i<5;i++){
        for (CharIndex = 0; NameList[i][0][CharIndex] != '¥0'; CharIndex++) {
            if(NameList[i][0][CharIndex] >= 'a' && NameList[i][0][CharIndex] <= 'z')
                NameList[i][0][CharIndex] += ('A' - 'a');
        }
        if(NameList[i][1][0]>='a' && NameList[i][1][0]<='z')
            NameList[i][1][0] += ('A' - 'a');

        for (CharIndex = 1; NameList[i][1][CharIndex] != '¥0'; CharIndex++) {
            if(NameList[i][1][CharIndex]>='A' && NameList[i][1][CharIndex]<='Z')
                NameList[i][1][CharIndex] += ('a' - 'A');
        }
        printf("%s %s¥n", NameList[i][1],NameList[i][0]);
    }
    return 0;
}
```

NameList[i][0]に姓がNameList[i][1]に名が入る

for文を使ってコンパクトに表現

姓をすべて大文字

名の先頭が英小文字
なら大文字

名の残りをすべて小文字

名→姓の順に出力

2. ファイル入出力

ファイル

- ・データのまとまりを表す単位の1つ
- ・一般的には、2次記憶装置上に保存されているので「電源が入っていない間も情報を保持する」特徴がある

ファイルを使ったプログラム

利点

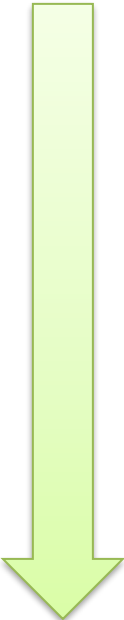
- ・処理するデータを毎回入力する必要がない
- ・計算結果を保存できる

欠点

- ・ファイル入出力が低速である場合が多い
- ・データファイルの管理が必要

2. ファイル入出力

Cでのファイルアクセスの流れ



① ファイルを開く

- データへのアクセスが可能な状態にする
- 他のプログラムからは書き換えが行えなくなる

② データへのアクセス

- ファイルからデータを読み取ったり書き込んだりする

③ ファイルを閉じる

- データへのアクセスを停止する
- 他のプログラムがファイルを利用することができるようになる

2. ファイル入出力

ファイルポインタ

ファイルのもつ様々な情報をまとめた変数(へのポインタ)
データの読み書きはファイルポインタを用いて行う



「ポインタ」となっているが、難しく考える必要はない。
ファイルをプログラム上で扱いやすくするためのものであり、
基本的には、ファイルポインタの詳しい中身について意識する
必要はない

プログラムとハードウェアとの仲介役のようなもの

2. ファイル入出力

このプログラムを実行して、ファイルが生成されることを確認しよう

例題：数値をファイルに書き込む

```
#include<stdio.h>
int main(void){
    int i;
    FILE *fp; /*ファイルポインタ*/

    fp = fopen("MyFile.txt","w");
    for(i=0;i<5;i++)
        fprintf(fp,"%d¥n",i);
    fclose(fp);
    return 0;
}
```

FILE *(変数名)」

ファイルを操作するための型(構造体)
→詳細を覚える必要はない。「ファイルを利用するときに宣言する必要がある」程度で良い。変数名はこのあとのファイルを操作する命令で用いる。
なお、「構造体」自身は第13回で説明

2. ファイル入出力

例題：数値をファイルに書き込む

```
#include<stdio.h>
int main(void){
    int i;
    FILE *fp; /*ファイルポインタ*/

    fp = fopen("MyFile.txt","w");
    for(i=0;i<5;i++)
        fprintf(fp,"%d¥n",i);
    fclose(fp);
    return 0;
}
```

ファイルの相対参照

パスにファイル名のみを書いた場合、プロジェクトのあるフォルダの中にあるファイルだけを探します

関数「fopen(ファイル名,形式)」

- ファイルを開き、データにアクセスするためのファイルポインタを返す関数
- 1番目の引数はファイル名(パス)を指す文字列
- 2番目の引数はファイルの形式
 - "w" : 書き込み(Write)
 - "r" : 読み出し(Read)
 - "a" : 追加書き込み(Append)
- 他にもテキスト形式(Text)やバイナリ形式(Binary)など、色々な形式がある

2. ファイル入出力

例題：数値をファイルに書き込む

```
#include<stdio.h>
int main(void){
    int i;
    FILE *fp; /*ファイルポインタ*/

    fp = fopen("MyFile.txt","w");
    for(i=0;i<5;i++)
        fprintf(fp,"%d¥n",i);
    fclose(fp);
    return 0;
}
```

関数「fprintf」

printfのファイルバージョン
1番目の引数にファイルポインタ
を追加する以外は、printfと同様

2. ファイル入出力

例題：数値をファイルに書き込む

```
#include<stdio.h>
int main(void){
    int i;
    FILE *fp; /*ファイルポインタ*/

    fp = fopen("MyFile.txt","w");
    for(i=0;i<5;i++)
        fprintf(fp,"%d¥n", i);
    fclose(fp);
    return 0;
}
```

関数「fclose」

ファイルを閉じるための関数
これ以降、fpを利用したファイル
アクセスはできなくなる

2. ファイル入出力

例題：数値をファイルに書き込む

```
#include<stdio.h>
int main(void){
    int i;
    FILE *fp; /*ファイルポインタ*/

    fp = fopen("MyFile.txt","w");
    for(i=0;i<5;i++)
        fprintf(fp,"%d¥n",i);
    fclose(fp);
    return 0;
}
```

実行結果

ソースファイルのあるフォルダに
「MyFile.txt」ができる

ファイルの内容

0
1
2
3
4

2. ファイル入出力

このプログラムを実行して、ファイルから値を読み取っていることを確認しよう

例題：値をファイルから読み込む

```
#include<stdio.h>
int main(void){
    int i,data[5];
    FILE *fp; /*ファイルポインタ*/

    fp = fopen("MyFile.txt","r");
    for(i=0;i<5;i++)
        fscanf(fp,"%d",&data[i]);
    fclose(fp);

    for(i=0;i<5;i++)
        printf("%d¥n",data[i]);
    return 0;
}
```

読み込みモード

関数「fscanf」

scanfのファイルバージョン
1番目の引数にファイルポインタ
を追加する以外は、scanfと同様

2. ファイル入出力

例題：値をファイルから読み込む

```
#include<stdio.h>
int main(void){
    int i,data[5];
    FILE *fp; /*ファイルポインタ*/

    fp = fopen("MyFile.txt","r");
    for(i=0;i<5;i++)
        fscanf(fp,"%d",&data[i]);
    fclose(fp);

    for(i=0;i<5;i++)
        printf("%d¥n",data[i]);
    return 0;
}
```

実行結果

画面出力

0
1
2
3
4

ここから演習課題です

3. 演習課題

演習11-1 追記処理の確認

```
#include<stdio.h>
int main(void) {
    int i;
    FILE *fp;    /*ファイルポインタ*/

    fp = fopen("MyFile.txt", "a");
    for(i=0; i<5; i++)
        fprintf(fp, "%d¥n", i);
    fclose(fp);
    return 0;
}
```

このとおりプログラムを実行し、ファイルに追記されていることを確認しよう

その後、再び"a"を"w"に変えて実行し、追記ではなく新規書込みになっていることを確認しよう

ここを"w"から"a"に変えた

3. 演習課題

演習11-2 2ファイル操作

このとおりプログラムを実行し、ファイル「MyFile2.txt」に配列aの内容が書き込まれていることを確認しよう
(※MyFile.txtに値が5つ以上書き込まれている必要があります)

```
#include<stdio.h>
int main(void){
    int i,a[5];
    FILE *fp1,*fp2;
    fp1 = fopen("MyFile.txt","r");
    fp2 = fopen("MyFile2.txt","w");

    for(i=0;i<5;i++)
        fscanf(fp1,"%d",&a[i]);
    for(i=0;i<5;i++)
        fprintf(fp2,"%d¥n",a[i]);
    fclose(fp1);
    fclose(fp2);
    return 0;
}
```

3. 演習課題

演習11-3 ファイル操作命令の練習

このプログラムは演習8-4[1]
(ベクトルをスカラー倍するもの)の解答例

```
#include <stdio.h>
#define N 10
void Scalar(int a[N], int k){
    int i;
    for(i=0; i<N; i++) a[i] = k * a[i];
}
int main(void){
    int i;
    int a[N], k=3;
    for(i=0; i<N; i++) a[i] = i;
    Scalar(a, k);
    for(i=0; i<N; i++) printf("a[%d]は%d¥n", i, a[i]);
    return 0;
}
```

実行結果をファイルに出力できるように書き換えよう.

書き換えるのはmain関数内.

→ファイルポインタ宣言, ファイルのオープン・クローズ, fprintf関数

3. 演習課題

発展11-1 点数の集計プログラム

ファイル「tensu.txt」の中に50人分の点数が書かれている
データを読み込み、「全体の平均点」、「全体の標準偏差」、「各人の偏差値」
を別ファイル「syukei.txt」に出力するプログラムを作成せよ

3. 演習課題

発展5-1 試験の統計処理プログラム

実行例

```
0番目の点数:72  
1番目の点数:68  
2番目の点数:94  
3番目の点数:26  
4番目の点数:73  
5番目の点数:88  
6番目の点数:58  
7番目の点数:85  
8番目の点数:92  
9番目の点数:45
```

```
試験結果(平均点=70.10点)  
0番:72点(偏差値 50.91)  
1番:68点(偏差値 48.99)  
2番:94点(偏差値 61.48)  
3番:26点(偏差値 28.81)  
4番:73点(偏差値 51.39)  
5番:88点(偏差値 58.60)  
6番:58点(偏差値 44.19)  
7番:85点(偏差値 57.16)  
8番:92点(偏差値 60.52)  
9番:45点(偏差値 37.94)
```



全体の平均点, 各人の偏差値
を出すプログラムは発展5-1で
実施済.

点数の入力をファイルから,
実行結果を別ファイルに
出力すれば良い.

【補足】文字列に対するfscanf (1/2)

例題：ファイルから文字列を読み込む

```
#include <stdio.h>

int main(void) {
    char pref[64], country[64];
    FILE *fp = fopen("input.txt", "r");
    fscanf(fp, "%63s %63s", pref, country);
    printf("国=%s, 都道府県=%s\n", country, pref);
    fclose(fp);
    return 0;
}
```

input.txt

Tokyo Japan

半角スペース

標準出力(ターミナル出力)

国=Japan, 都道府県=Tokyo

【補足】文字列に対するfscanf (2/2)

例題: ファイルからタブ区切りの文字列(空白含む)を読み込む

```
#include <stdio.h>

int main(void) {
    char name[64], address[64];
    FILE *fp = fopen("input2.txt", "r");
    fscanf(fp, "%63[^\\t]\\t%63[^\\n]", name, address);
    printf("名前=%s\\n所属=%s\\n", name, address);
    fclose(fp);
    return 0;
}
```

※ 補足

%[^\\t]

タブ文字以外の
文字を読み込む

%[^\\n]

改行文字以外の
文字を読み込む

input2.txt

農工 太郎 東京農工大学 小金井キャンパス

← タブ[Tab] →
← 半角スペース →

標準出力(ターミナル出力)

名前=農工 太郎
所属=東京農工大学 小金井キャンパス

本日の説明は以上です。
続いて、第2回目レポート課題について説明をします。
説明後に本日の演習課題に取り組んでください。